King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Term 1

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 5

Setup Instructions: Isabelle PIDE MCP and Coding Agents In this tutorial, we explore AI-assisted theorem proving, interactive proof development, and automated reasoning in Isabelle/HOL.

To enable an AI coding agent to interact directly with your Isabelle environment:

1. **Clone PIDE MCP**: Clone the repository from https://github.com/kappelmann/isabelle-pide-mcp and switch to branch `Isabelle2025-2`:

   ```
   git clone -b Isabelle2025-2
   https://github.com/kappelmann/isabelle-pide-mcp
   ```

2. **Register Component**: Follow the instructions in the repository's `README.md` to register the MCP server as an Isabelle component:

   ```
   isabelle components -u <path-to-isabelle-pide-mcp>
   ```

   Please read the `README.md` carefully for full setup and agent configuration options.

3. **Recommended Coding Agent**: We recommend using **OpenCode** (https://opencode.ai), an open-source terminal coding agent supporting MCP. OpenCode connects to free model tiers (such as `opencode/big-pickle`, Google Gemini 2.5 Flash / Flash-Lite, or local models via Ollama), allowing you to complete the exercises completely free of charge.

4. **Building Skill Files & Agent Definitions**: In OpenCode, you can define specialized **skills** and **subagents**:

   - *Skills*: Stored in `.opencode/skills/<name>/SKILL.md` or `~/.agents/skills/<name>/SKILL.md` with YAML frontmatter (`name`, `description`) followed by instructions.
   - *Subagents*: Stored in `.opencode/agents/<name>.md` or `~/.config/opencode/agents/<name>.md` with frontmatter specifying `mode: all`, `model`, and a dedicated system prompt.

5. **The Four Agent Proving Roles**: As discussed in the lectures on agentic proving architectures, complex formal proofs benefit from coordinating four specialized roles:

   a) **Planner / Sketcher Agent**: High-level architectural reasoning, decomposing main goals, discovering auxiliary invariant lemmas, and sketching Isar proof skeletons with `sorry`.

   b) **Prover / Coder Agent**: Tactical execution, expanding equations, and filling routine subgoals.

c) **Repairer Agent**: Interactive compiler feedback loop via `isabelle-pide-mcp`, inspecting goal states, diagnosing type/syntax errors and failed tactics, generalizing variables, and eliminating `sorry`.

d) **Critic / Sledgehammer Agent**: Soundness evaluation, circularity checks, and invoking automated provers (E, Vampire, Z3, CVC4) to compress multi-line proofs into single-line automated steps.

Exercise 5.1 Recursive Summation and Counterexample Finding

Define a recursive function *sum_upto* :: *nat* ⇒ *nat* that calculates the sum of the first n natural numbers:

$$\text{sum\_upto}(n) = \sum_{i=1}^{n} i = 1 + 2 + \cdots + n$$

such that $\text{sum\_upto}(0) = 0$ and $\text{sum\_upto}(n+1) = (n+1) + \text{sum\_upto}(n)$.

fun *sum_upto* :: *nat* ⇒ *nat* **where**

Before trying to prove a theorem, it is good practice to test conjectures with automated counterexample generators such as *quickcheck* and *nitpick*. Consider the erroneous conjecture:

$$\text{sum\_upto}(n) = n \cdot (n+1)$$

In Isabelle, one can formulate this conjecture and run *quickcheck* or *nitpick* to automatically find a counterexample (e.g., for $n = 1$, $\text{sum\_upto}(1) = 1 \neq 2$).

Exercise 5.2 Draft, Sketch, and Prove (DSP): Gauss Summation Formula

Gauss' famous summation formula states that:

$$2 \cdot \text{sum\_upto}(n) = n \cdot (n+1)$$

In the two-stage "Draft, Sketch, and Prove" (DSP) architecture used by modern AI proving agents, one first drafts a high-level proof sketch with intermediate goals marked with *sorry*, verifying that the structural flow of the proof is sound, before completing each individual step.

Prove this formula by induction on n using a structured Isar proof with explicit equational reasoning (*also have ... finally show ?case*).

lemma *sum_upto_gauss*: *2 * sum_upto n = n * (n + 1)*

Exercise 5.3 Proof Automation and Sledgehammer

While the step-by-step equational proof in the previous exercise makes every algebraic transformation explicit, automated reasoning tools can dramatically shorten proof construction.

Use Isabelle's automated proof tools (such as *sledgehammer* or the simplifier with algebraic simplification rules) to prove the Gauss summation theorem in a single line.

lemma *sum_upto_gauss_auto*: *2 * sum_upto n = n * (n + 1)*

Exercise 5.4 Accumulator-Based Tail-Recursive Summation

In Week 3, we discussed accumulator generalisation for list recursion. The same technique applies to numerical recursion.

Define a tail-recursive function *sum_upto_acc* :: *nat ⇒ nat ⇒ nat* that accumulates the running sum in an extra argument:

$$\text{sum\_upto\_acc}(0, \text{acc}) = \text{acc} \quad \text{and} \quad \text{sum\_upto\_acc}(n{+}1, \text{acc}) = \text{sum\_upto\_acc}(n, n{+}1{+}\text{acc})$$

fun *sum_upto_acc* :: *nat ⇒ nat ⇒ nat* **where**

Prove the invariant generalisation lemma relating *sum_upto_acc* to *sum_upto*:

$$\forall \text{acc}.\ \text{sum\_upto\_acc}(n, \text{acc}) = \text{sum\_upto}(n) + \text{acc}$$

Use induction on n with an arbitrary accumulator *acc*.

lemma *sum_upto_acc_eq*: *sum_upto_acc n acc = sum_upto n + acc*

Conclude that the tail-recursive summation starting with an accumulator of 0 satisfies the closed-form Gauss formula.

lemma *sum_upto_acc_gauss*: *2 * sum_upto_acc n 0 = n * (n + 1)*

Exercise 5.5 Multi-Agent Proof Engineering: Stack Machine Compiler Correctness

A classic benchmark in verified compilation is the correctness of an expression compiler targeting a stack machine. Consider an expression datatype supporting integer constants, addition, and multiplication:

datatype *exp = Val int | Plus exp exp | Mult exp exp*

datatype *instr = PUSH int | ADD | MULT*

fun *eval* :: *exp ⇒ int* **where**
  eval (Val n) = n
| *eval (Plus e1 e2) = eval e1 + eval e2*
| *eval (Mult e1 e2) = eval e1 * eval e2*

The stack machine executes a sequence of instructions on an integer stack:

```
fun exec :: instr list ⇒ int list ⇒ int list where
  exec [] s = s
| exec (PUSH n # ins) s = exec ins (n # s)
| exec (ADD # ins) (n2 # n1 # s) = exec ins ((n1 + n2) # s)
| exec (ADD # ins) _ = []
| exec (MULT # ins) (n2 # n1 # s) = exec ins ((n1 * n2) # s)
| exec (MULT # ins) _ = []
```

The compiler compiles an expression into a list of postfix instructions:

```
fun compile :: exp ⇒ instr list where
  compile (Val n) = [PUSH n]
| compile (Plus e1 e2) = compile e1 @ compile e2 @ [ADD]
| compile (Mult e1 e2) = compile e1 @ compile e2 @ [MULT]
```

Task A: Direct Proof Failure with a Single LLM Agent The target correctness theorem states:

$$\text{exec}(\text{compile}(e), []) = [\text{eval}(e)]$$

1. Attempt to prove this theorem directly in one shot using OpenCode with a free model tier (e.g., `opencode/big-pickle` or Gemini Flash) via the Isabelle PIDE MCP server.
2. **Observe and document the failure**: Direct structural induction on e fails because `compile` concatenates instruction sequences with `@`, whereas `exec` is defined recursively on instruction lists. Without an append-distribution or continuation lemma, the induction hypothesis cannot be applied to recursive subexpressions. A single-shot LLM typically gets stuck in infinite repair loops, generates invalid Isar scopes, or hallucinates non-existent lemmas.

Task B: Skill Engineering for Multi-Agent Proving Now, build two cooperating skill files (or subagent definitions) in OpenCode:

1. **Planner Skill (`isabelle-planner`)**: Creates a proof plan by discovering the necessary generalized invariant:

   $$\text{exec}(\text{compile}(e)@\text{ins}, s) = \text{exec}(\text{ins}, \text{eval}(e)\#s)$$

   The planner drafts this lemma with `sorry`, and completes the proof of `compiler_correct` by instantiating the helper lemma with empty lists (`ins = []`, `s = []`). *Important*: Name the instruction stream variable `ins` or `ils`; do **not** name it `is`, because `is` is a reserved Isar keyword!
2. **Repairer Skill (`isabelle-repairer`)**: Connects to `isabelle-pide-mcp` to inspect compiler diagnostics and remaining subgoals. The repairer reads the sorried lemma, supplies the generalization variables (`induction e arbitrary: ins s`), handles algebraic properties, and verifies that the proof reaches 0 errors and eliminates all `sorry` statements.

Verify that chaining your Planner agent followed by your Repairer agent successfully solves the theorem!

lemma *exec_compile*: *exec* (*compile e* @ *ins*) *s* = *exec ins* (*eval e* # *s*)
theorem *compiler_correct*: *exec* (*compile e*) [] = [*eval e*]